

Assignment -3

Section-A

1) What is race condition?

Ans A race condition is a situation in which the behaviour of a computer program or system depends on the relative timing of events, leading to unpredictable and undesirable outcomes.

2) Why lock variables are used?

Ans Lock variables are used in process synchronization to provide mutual exclusion and ensure that only one process can access a shared resource or critical section at a time, preventing data corruption and race conditions.

3) What is critical region?

Ans A critical region is a section of code in a program where shared resources are accessed and it needs to be protected using synchronization mechanisms like locks to avoid conflicts and ensure data integrity.

4) What is Blocking send in message passing model?

Ans In a message passing model blocking

send refers to a situation where the sender process is suspended until the receiver acknowledges the receipt of a message. It ensures that the sender does not proceed until the message is safely delivered.

5) What is the use of claim edge in Resource allocation graph.

Ans Claim edge in a Resource allocation graph is used to indicate that a process may request additional resources in the future. It allows the system to track potential resources needs and avoid deadlock situations.

Section - B

1) What is semaphore? Explain it.

Ans A semaphore is a synchronization tool used in operating systems and concurrent programming to control access to shared resources.

2) It is essentially an integer variable that can be accessed through two standard atomic operations:-

wait(S)	Signal(S)
while (S <= 0);	S = S + 1;
S = S - 1;	

3) wait(S) operation waits until the semaphore S is greater than or equal to zero. If S is negative, it enters a busy-wait loop until 'S' becomes non-negative and then it decrements 'S'. This operation ensures that process are delayed when the resource is in use.

4) signal(S) operation increments the semaphore 'S'. This operation is used to release the resource or signal other processes that they can proceed.

5) They are essential for managing critical section to shared resources among multiple processes. They are used to prevent race condition & ensure that only one process access a shared resource at a time.

6) No two processes can simultaneously modify the same semaphore value, ensuring that access to shared resources is properly coordinated. It is the critical aspect of semaphore that the modification to the semaphore value in wait() & signal() operations must be executed automatically.

7) The semaphore format is given by:-
wait (Mutex);
Critical Section;

Signal (mutex);
Remainder;

Q7 There are 2 types of semaphore:-

→ Binary (mutex locks) :- values restricted to 0 & 1 and they are often used for mutual exclusion.

→ Counting :- values over an unrestricted domain $-\infty$ to $+\infty$ & can be employed to control access to a finite number of resources instances.

Q8 What is difference between deadlock prevention & avoidance.

Ans	Deadlock Prevention	Deadlock Avoidance
Objective	To prevent one of the four necessary conditions for deadlock from occurring: mutual exclusion, hold & wait, no preemption & circular wait.	To manage resource allocation dynamically to ensure that the system can enter a safe state & avoid potential deadlocks.

Approach	Enforces restrictions on how processes request and release resources.	Uses algorithms to analyze the current resource allocation state and make resource request based on safety.
----------	---	---

System state	Changes the system state to eliminate one of the conditions.	Monitors the resource allocation state & decides resources requests dynamic.
--------------	--	--

required for deadlock to happen

ally to prevent unsafe states

Flexibility - May impose restrictions on process behavior, leading to underutilization of resources.

Complexity - Typically simpler to implement but may limit system efficiency.

Resource Utilization - May result in inefficient resource utilization as it may not allow processes to use resources if no deadlock is imminent.

Eg :- Resource allocation graph, request release protocols, limited resource access.

More flexible as it adapts to changing resource requirements and avoids deadlocks without imposing strict restrictions.

More complex as it requires resource allocation algorithms like Banker's algorithms.

Tends to utilize resources more efficiently by allowing processes to proceed when it's safe.

Banker's algorithm, wait-die & wound-wait schemes.

Q9 Explain the Precedence graph with an Example.

Ans :- Precedence graph is a directed acyclic graph which is used to show the execution level of several processes in operating system.

2. It consists of nodes and edges. Nodes represent the process and edges represent flow of execution.

Properties:-

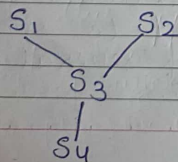
3. Directed graph:- Nodes are connected by directed edges.

4. Acyclic graph:- It contains no cycles. loop i.e., you cannot follow a path of directed edges that starts & ends at the same node.

5. Nodes correspond to program statements:- Each node in the graph represents an individual statement or instruction in the program code.

6. Execution order:- directed edges between nodes represent the execution order of the program statements.

7. Example:- $S_1: a = x + y$
 $S_2: b = z + 1$
 $S_3: c = a - b$
 $S_4: w = c + 1$



S_1 & S_2 should complete before S_3 starts.

And S_3 should complete for execution of S_4 .

Section-C

1. What is Producer consumer problem? Give the solution with sleep & wakeup approach.

Ans. The Producer-Consumer problem is a classic synchronization problem in computer science. It involves two types of processes, producers & consumers, who share a common finite-sized buffer or queue.

⇒ The problem is to ensure that:-

1. Producers don't produce data into a full buffer.

2. Consumers don't consume data from an empty buffer.

3. Producers & consumers don't access the buffer simultaneously, which could lead to data corruption.

4. Inconsistency due to the sharing of common variable.

5. Printer spoiler problem.

⇒ One common solution to the Producer consumer problem uses the "sleep and wakeup" approach with semaphore or conditional variable.

1) Initialize the buffer, semaphores and other necessary variables.

2) Implement producer-consumer functions-

Producer

a) Wait for an empty slot in buffer (using SP)

b) Once available, produce data & add it to buffer.

c) Signal the semaphore to indicate that the buffer is no longer empty.

Consumer

a) Wait for a data item in the buffer.

b) Once available consume the data & remove it from buffer.

c) Signal the SP to indicate that a slot in buffer is now free.

3) Properly initialize & manage the SP to control access to the buffer.

4) Start multiple producer & consumer threads.

→ In this approach producers & consumers sleep when they can't perform their task (eg- if buffer is full for producers or empty for consumers) & they wake up

when conditions change (space become available or data is added.)

2) What is deadlock detection methods for single instance and multiple instance of resources?

Ans Deadlock detection methods are technique use to identify the presence of deadlocks in a system. Deadlocks can occur in system with resource allocation, where processes or threads are waiting for resources that are currently held by other processes. There are different methods for detecting deadlocks in systems with single instances of resources:- and

Single instance resources :- For systems with single, instances of resources (each resource exists in only one copy), one common method of deadlock detection is using a Resource Allocation Graph (RAG). Here's how it works

- Process :- Represent as nodes in the graph.
- Resources :- Represent as rectangles in the graph.
- Edges :- Two types of edges:- request and assignment.
- A request edge from a process to a resource indicates that the process is requesting the resources.

- An assignment edge from a resource to a process indicates that the resource is assigned to that process.

2. Multiple Instance Resources :- In systems with multiple instances of resources (multiple copies of the same resource) a variant of the Banker's algorithm can be used for deadlock detection.

These methods help detect deadlocks in both single and multiple instance resource scenarios. Once a deadlock is detected actions can be taken to resolve it such as killing one or more resources or releasing resources.

3. Write banker's algorithm and give the solution for given state.

Process	Maximum				Allocated				Needed				Available			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
0	6	0	1	2	4	0	0	1	2	0	1	1	3	2	1	1
1	1	7	5	0	1	1	0	0	0	6	5	0				
2	2	3	5	6	1	2	5	4	1	1	0	2				
3	1	6	5	3	0	6	3	3	1	0	2	0				
4	1	6	5	6	0	2	1	2	1	4	4	1				

Ans - Banker algorithm is a deadlock avoidance algorithm that is used to ensure that resource allocation requests from processes in an operating system won't

lead to a deadlock.

→ It has 2 states
 ↳ Safe state
 ↳ Unsafe state

→ If there is a deadlock, it means the state is unsafe state.

→ If there is a unsafe state, it means it may or may not be the condition of deadlock.

$$\text{Remaining Need} = \text{Max}^{\circ} \text{Need} - \text{Current allocation}$$

Process	Max-Need	Current Allocation	Current Available	Remaining Need
	A B C D	A B C D	A B C D	A B C D
P0	6 0 1 2	4 0 0 1	3 2 1 1	2 0 1 1
P1	1 7 5 0	1 1 0 0	7 2 1 2	0 6 5 0
P2	2 3 5 6	1 2 5 4	8 4 6 6	1 0 2
P3	1 6 5 3	0 6 3 3	8 10 9 9	0 2 0
P4	1 6 5 6	0 2 1 2	8 12 10 12	1 4 4 4

Execution :- P0, P2, P3, P4, P1.